

Loan Default Prediction

Megang Nkanga Junile Staures

Kaunas University of Technology, Kaunas, Lithuania

Machine Learning in Industry – Days 1–2

Cardo AI / MSCA Digital Doctoral Network

Training data: `d2assignment_dataset.csv.gz` – 96,112 rows $\times$ 54 columns (`row_id` + 52 features + `target`)

Test data: `d2assignment_test.csv.gz` – 23,888 rows $\times$ 53 columns (`row_id` + 52 features, no `target`)

1 Data audit

1.1 Target

13,797 defaults out of 96,112 loans (14.4%); Figure 1. Two consequences: raw accuracy is close to meaningless (predicting "no default" for everyone scores 86%), and the default 0.5 threshold is a poor operating point unless corrected (Section 4.5).

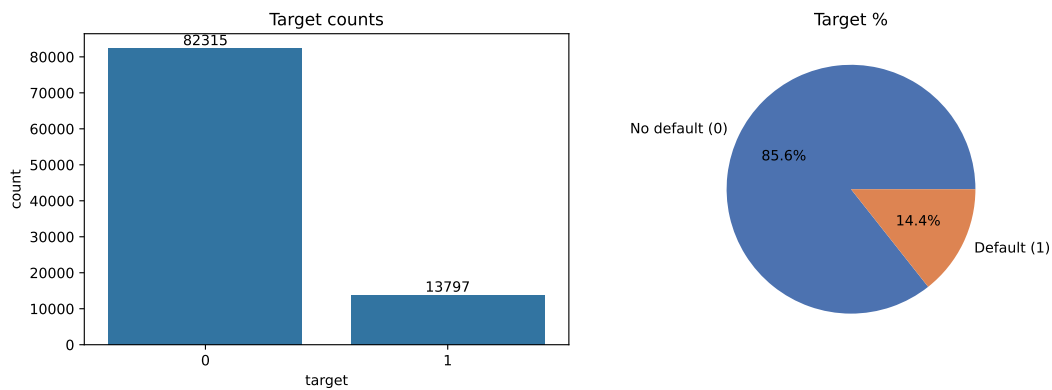


Figure 1: Target distribution

1.2 Missingness

Concentrated in a few columns, and consistent between train and test:

Column	% missing (train)	% missing (test)
m_since_delinq	48.34%	47.65%
m_since_inquiry	10.25%	10.99%
emp_years	6.22%	7.40%
card_util	1.08%	1.13%
pct_cards_hi_util	1.07%	1.19%
card_headroom	0.99%	1.08%
m_since_card_open	0.94%	1.04%
revolving_util	0.06%	0.03%
avg_balance	2 rows	0 rows

1.3 Post-origination leakage

Nine columns are 100% missing in test but fully populated in train: `account_flag`, `fee_adj`, `last_txn_amt`, `late_fees`, `payments_total`, `principal_rcv`, `residual_amt`, `review_gap_m`, `score_recent`. All describe post-origination servicing events – unknowable at the moment a prediction is actually needed.

Correlation with target confirms this: in the raw data these nine columns dominate (`score_recent` = -0.575 , `residual_amt` = 0.506 , `fee_adj` = 0.495 , `principal_rcv` = -0.330 , `account_flag` = 0.324 , `review_gap_m` = -0.260 , `payments_total` = -0.258 , `last_txn_amt` = -0.251). After dropping them, the strongest remaining predictors are `pricing_rate` = 0.205 and `risk_band` = 0.204 – an order of magnitude weaker, and plausibly genuine (lender’s own risk assessment) rather than leaked.

Action: all nine dropped. Figures 2 and 3 show the before/after.

1.4 Constant column

`app_kind` is constant at 0 across all 96,112 training rows; in test it takes 0 (23,830 rows) and 1 (58 rows). Zero training variance means zero learnable signal, so it was dropped; to avoid a train/test mismatch rather than for leakage reasons.

After dropping the nine leakage columns and `app_kind` together: **train** $96,112 \times 44$ (`row_id` + 42 features + target); **test** $23,888 \times 43$ (`row_id` + 42 features).

1.5 Train/test distribution shift

Column-wise Kolmogorov–Smirnov tests. Largest shifts:

Feature	KS statistic	<i>p</i> -value
pricing_rate	0.132	3×10^{-293}
income_verif	0.064	2×10^{-68}
risk_band	0.053	4×10^{-47}
pct_cards_hi_util	0.047	4×10^{-36}
revolving_util	0.046	2×10^{-35}

p-values are tiny mainly because of sample size; the KS statistics themselves are small (all but the top five below 0.05).

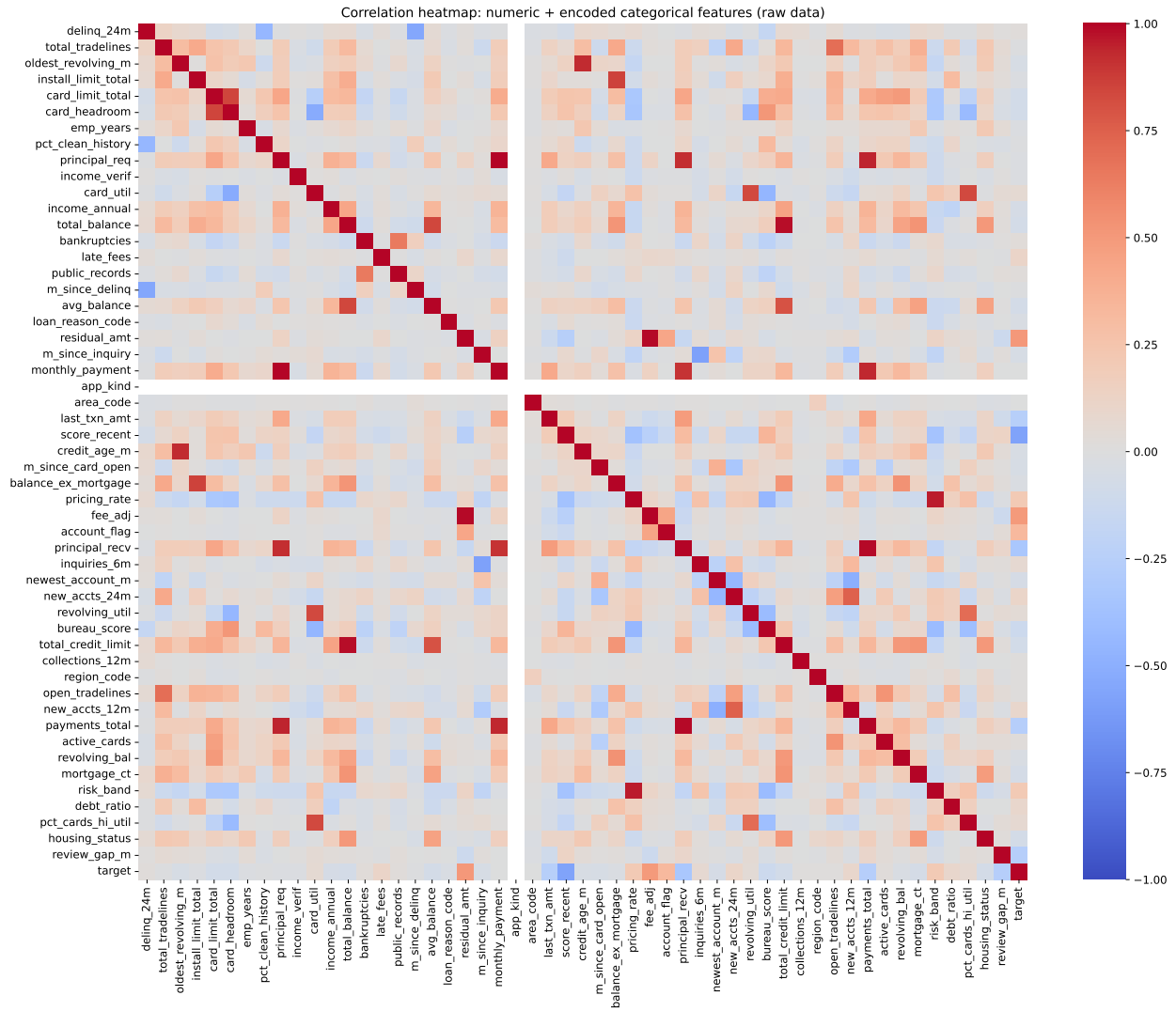


Figure 2: Feature–target correlations, raw data.

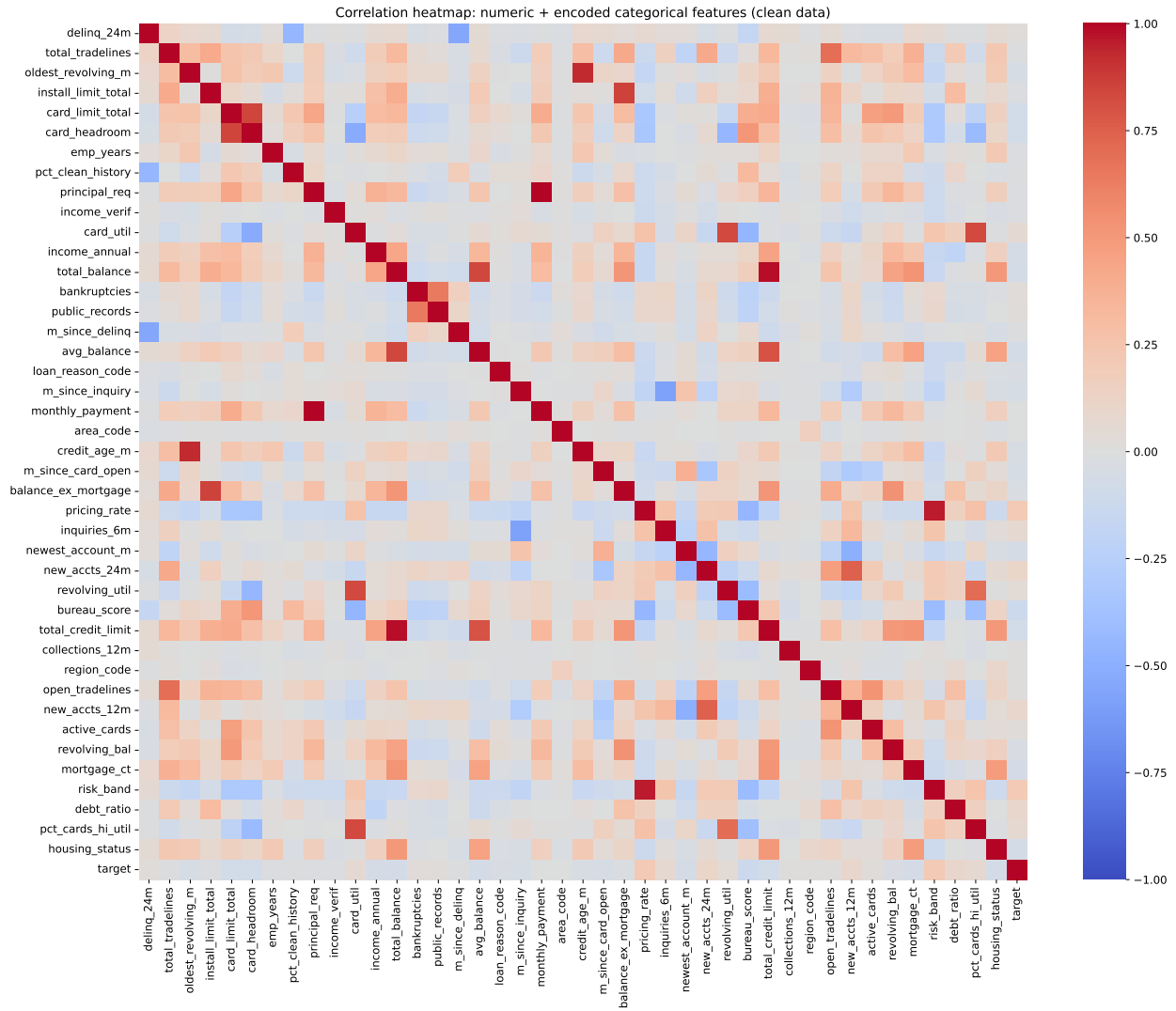


Figure 3: Feature–target correlations, after removing the nine leakage variables.

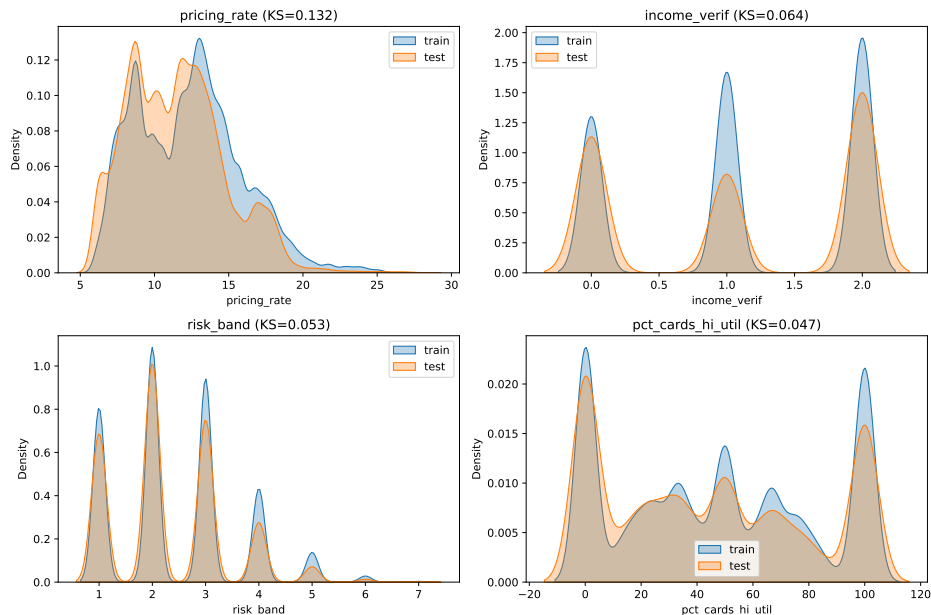


Figure 4: Largest train/test distribution shifts.

Conclusion: train and test are broadly similar; mild drift, mainly in `pricing_rate`, `income_verif`, and `risk_band`. No structural shift severe enough to require a different validation scheme (e.g. time-based split), but worth watching when interpreting validation AUC.

1.6 Is missingness informative?

Default rate, missing vs. present:

Feature	% missing	Rate if missing	Rate if present	Diff
<code>emp_years</code>	6.22%	20.93%	13.92%	+7.0 pts
<code>revolving_util</code>	0.06%	20.37%	14.35%	+6.0 pts
<code>m_since_inquiry</code>	10.25%	9.61%	14.90%	-5.3 pts
<code>m_since_card_open</code>	0.94%	15.60%	14.34%	+1.3 pts
<code>m_since_delinq</code>	48.34%	13.77%	14.90%	-1.1 pts

(`avg_balance` shows a large gap too, but only 2 rows are missing – not reliable.) Missingness is not MCAR, so it carries signal. **Action:** binary indicators created for the four features above; underlying values then median-imputed (median from training data only).

2 Preprocessing and feature decisions

Decision	What	Why
Drop 9 servicing columns	Listed in 1.3	100% missing in test; post-origination leakage
Drop app_kind	Dropped	Constant in train, zero signal
Missing-value indicators	4 binary flags (<code>emp_years</code> , <code>m_since_inquiry</code> , <code>m_since_delinq</code> , <code>revolving_util</code>)	Missingness correlates with default rate
Median imputation	All remaining numeric columns with gaps	Robust to outliers; fitted on train only
Categorical typing	<code>housing_status</code> , <code>income_verif</code> , <code>loan_reason_code</code> , <code>risk_band</code> , <code>region_code</code> , <code>area_code</code> cast to category	Coded/discrete fields, not continuous
One-hot encoding	<code>pd.get_dummies(columns=cat_col, drop_first=True)</code> , after aligning each column's category list across train/test	Used as the single, final representation for all five models, including CatBoost, for a consistent comparison

3 Validation design

Stratified 80/20 train/validation split (`random_state=42`), preserving the $\approx 14.4\%$ default rate. Model comparison used stratified 5-fold then 10-fold CV; hyperparameter search and the follow-up experiments (Section 4.6) used 10-fold CV throughout, for consistency with the model comparison. Given the mild train/test drift found in Section 1.5, a stratified split from the same population is a reasonable proxy for out-of-sample performance.

Two caveats: (1) `pricing_rate` has both the largest drift ($KS = 0.132$) and the top SHAP importance, so validation AUC could be a slight overestimate; (2) with 983 one-hot columns from $\approx 77,000$ rows, rare categorical levels could look predictive by chance without generalising.

Runtime: the 10-fold hyperparameter search (30 iterations) and the 10-fold follow-up experiments were the dominant cost of this pipeline, together taking well over an hour on a standard laptop CPU. This was a deliberate trade-off in favour of a more thorough, consistent-fold-count comparison across every experiment, at the cost of iteration speed.

Pre-registered expectation: test ROC-AUC in the 0.69–0.70 range, based on the 10-fold CV mean of 0.7001 and held-out validation ROC-AUC of 0.6953.

4 Modeling

4.1 Majority-class baseline

Before any model: predicting the majority class (no default) for every validation row gives **86.5% accuracy** but **ROC-AUC of exactly 0.5**; a constant prediction cannot rank rows relative to each other. This is the concrete illustration behind the earlier claim that accuracy

is close to meaningless here (Section 1): a model that learns nothing at all already looks 86% "accurate". Every model below is judged on AUC against this 0.5 floor, not on accuracy.

4.2 Model comparison

Model	5-fold Mean	5-fold SD	10-fold Mean	10-fold SD
Logistic Regression	0.6735	0.0046	0.6759	0.0058
Random Forest	0.6920	0.0037	0.6935	0.0047
LightGBM	0.6926	0.0039	0.6932	0.0041
XGBoost	0.6977	0.0027	0.6979	0.0036
CatBoost	0.6995	0.0026	0.7001	0.0040

Tree-based models beat the linear baseline by ≈ 1.6 -2.4 AUC points; among tree models the gap is under 0.7 points. CatBoost wins consistently and was carried forward.

4.3 Hyperparameter tuning

RandomizedSearchCV (30 iterations, 10-fold CV, ROC-AUC) over `iterations` $\in \{200, 300, 500, 700\}$, `depth` $\in \{4, \dots, 8\}$, `learning_rate` $\in \{0.01, 0.03, 0.05, 0.1\}$, `l2_leaf_reg` $\in \{1, 3, 5, 7, 9\}$. Best configuration:

- `learning_rate` = 0.03
- `l2_leaf_reg` = 3
- `iterations` = 700
- `depth` = 8

Mean 10-fold ROC-AUC: **0.7008**. This represents a small improvement over the untuned CatBoost baseline (0.7001).

4.4 Final validation & SHAP

Held-out validation ROC-AUC: **0.6953** – ≈ 0.005 below the 10-fold mean, within normal fold variability (no overfitting evidence).

Top SHAP drivers (Figures 5, 6): higher `pricing_rate`, `new_accts_24m`, `debt_ratio` raise predicted risk; higher `bureau_score`, `income_annual`, `total_credit_limit` lower it – consistent with standard credit-risk intuition. `emp_years_missing` ranks in the top 20, supporting the decision to keep missingness as its own signal.

4.5 Threshold selection

At 0.5, recall is 0.005; the model flags almost no defaulters despite $\text{AUC} \approx 0.70$. A threshold sweep gives the F1-optimal point at **0.15**: precision 0.23, recall 0.64, F1 0.34 (2,759 defaults / 16,464 non-defaults; accuracy 0.64).

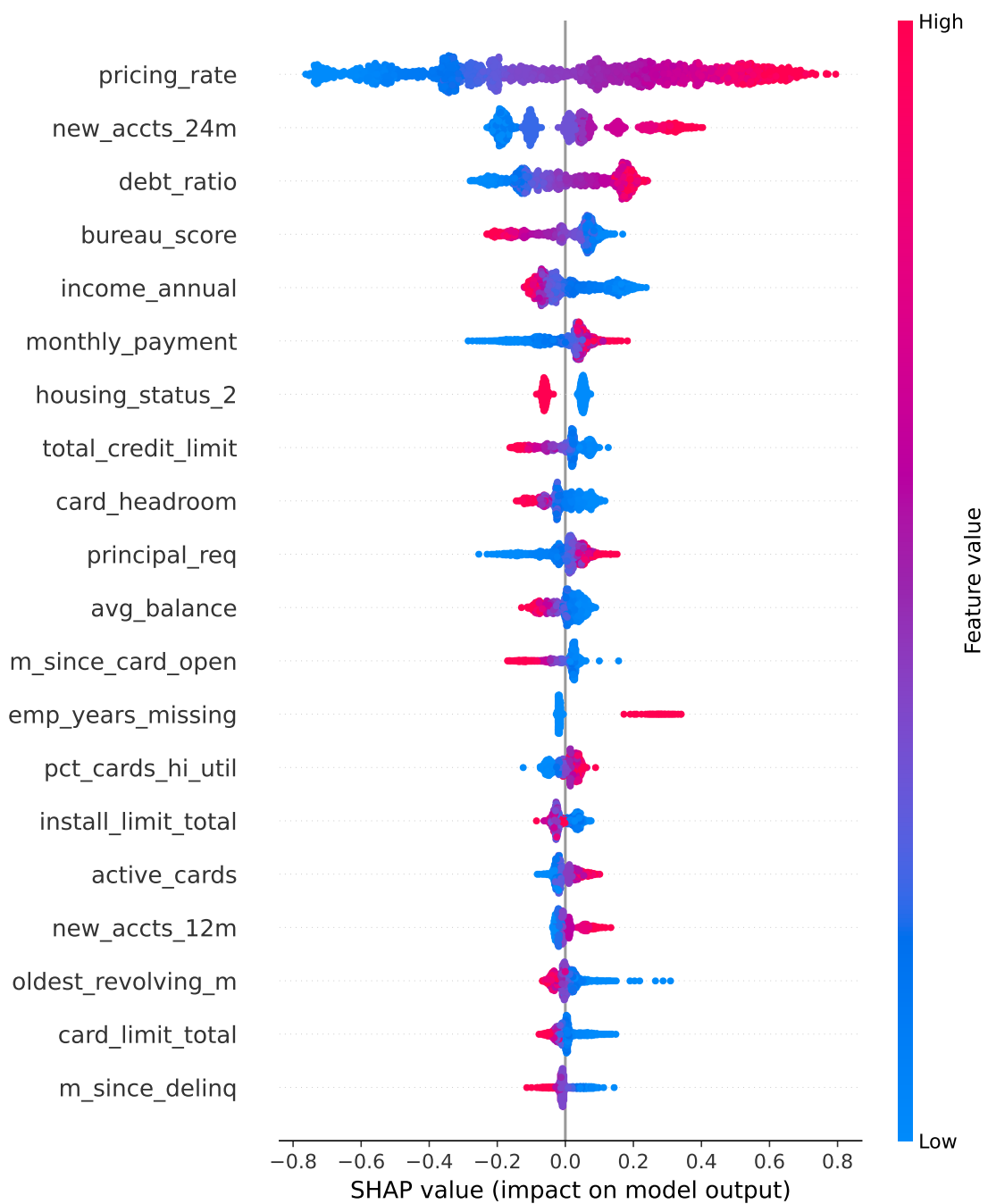


Figure 5: SHAP summary plot.

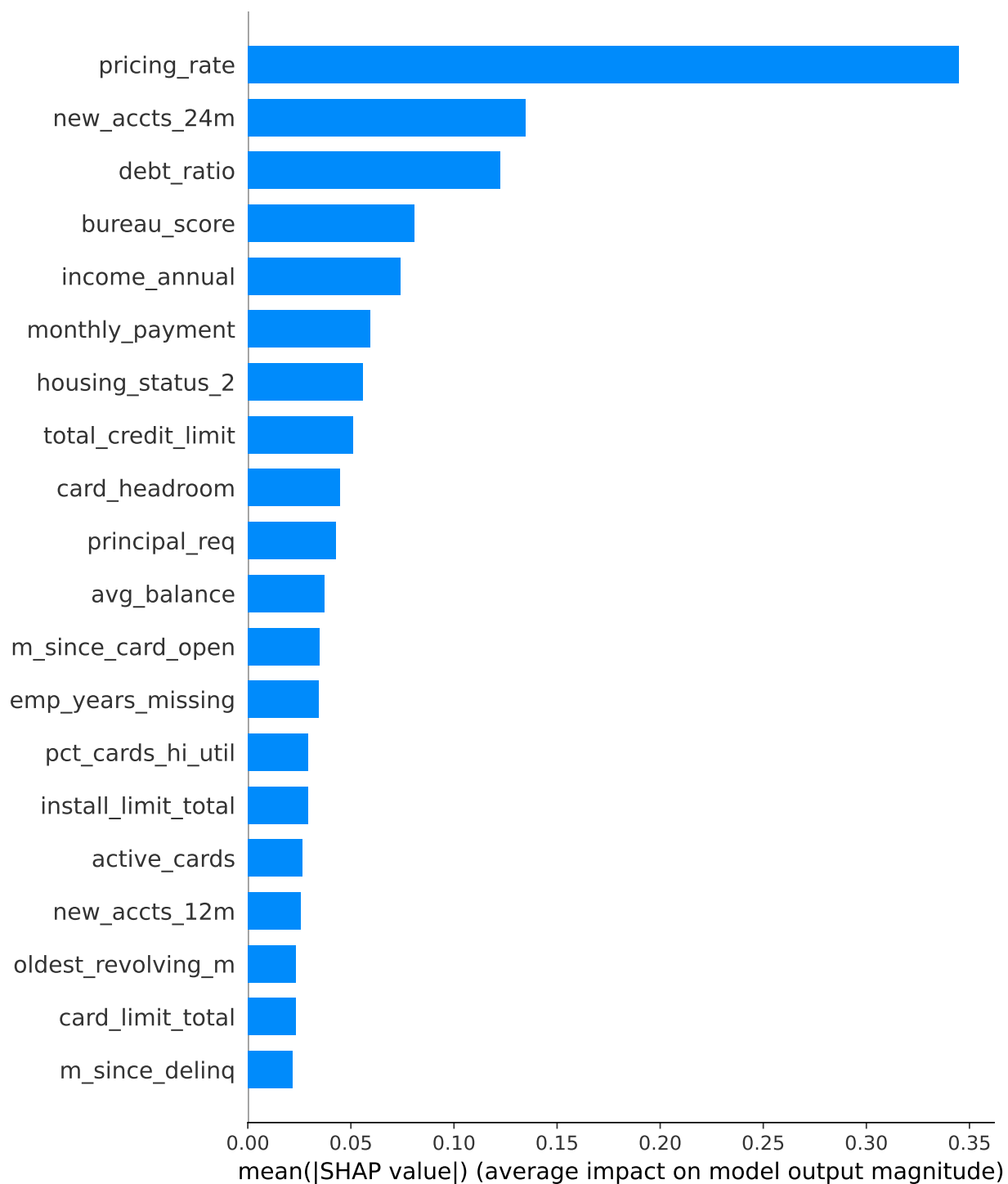


Figure 6: Mean |SHAP| feature importance.

Threshold	Precision	Recall	F1
0.05	0.162	0.971	0.278
0.10	0.196	0.832	0.318
0.15	0.230	0.640	0.338
0.20	0.261	0.438	0.327
0.25	0.300	0.277	0.288
0.30	0.324	0.150	0.206
0.40	0.361	0.030	0.055
0.45	0.338	0.010	0.019
0.50	0.448	0.005	0.009

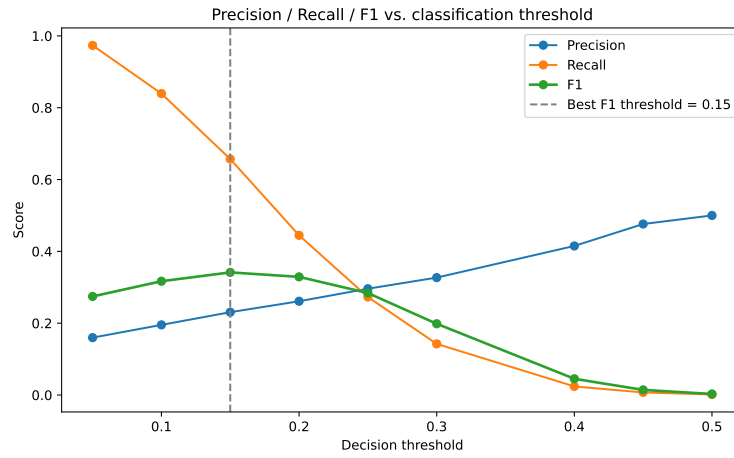


Figure 7: Precision/recall/F1 vs. threshold.

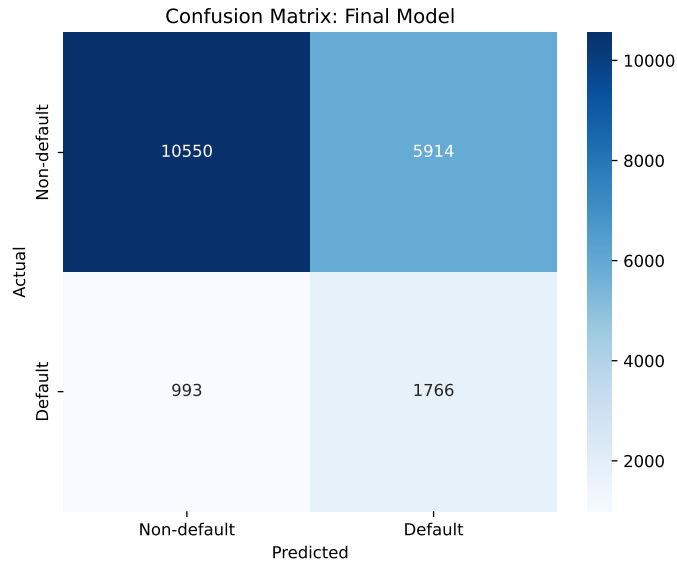


Figure 8: Confusion matrix at threshold 0.15 (TP 1,766 / FN 993 / FP 5,914 / TN 10,550).

4.6 Follow-up experiments

Both use the same `skf_10` folds as the baseline comparison.

SMOTE (fit within each fold only; verified via `imblearn.Pipeline` and a manual loop, matching to 4 decimals): AUC drops from 0.7001 to **0.6945** (SD 0.0053).

Rejected. The dataset is moderately imbalanced (14.4% defaults), but the baseline CatBoost model already achieved the highest ROC-AUC without resampling. Introducing SMOTE consistently reduced performance, suggesting that synthetic oversampling did not improve the model’s ability to rank borrowers by default risk.

Class weighting.

Because defaults represent only 14.4% of observations, class-weighted variants of CatBoost and XGBoost were tested.

Model	10-fold ROC-AUC
CatBoost	0.7001
CatBoost Balanced	0.6996
XGBoost	0.6979
XGBoost Weighted	0.6970

Both weighting strategies slightly reduced ROC-AUC. Consequently, the unweighted CatBoost model remained the final choice.

5 Final submission

Tuned CatBoost (depth=8, iterations=700, l2_leaf_reg=3, learning_rate=0.03) refit on the full labeled data (96,112 rows $\times$ 983 features, train + validation combined), then used purely for inference on the 23,888-row test set (whose true labels were not available during development).

The held-out validation predictions (`val_submission.csv`) were independently rescored against `validation_labels.csv` using `score_submission.py`, returning **ROC-AUC 0.6953**.

`submission.csv` itself was validated against the assignment’s format requirements (23,888 rows, no duplicate `row_id`, probabilities in $[0, 1]$); its AUC cannot be computed directly, since the test set’s true labels are not available.

Prediction distribution:

- Mean: 0.1314
- Standard deviation: 0.0867
- Minimum: 0.0084
- Maximum: 0.6146

No probabilities were exactly 0 or 1. The mean prediction is broadly consistent with the observed 14.4% training default rate.

6 Limitations

At threshold 0.15, precision is 0.23; roughly three-quarters of flagged loans are false positives. AUC ≈ 0.70 is moderate, not strong, discrimination. Best used as a screening tool for human review, not automatic rejection.